Solution Architecture - ShopEZ Stock Trading Platform

Parameter	Details
Date	18 June 2026
Team ID	SE-09
Project Name	ShopEZ Stock Trading Platform
Maximum Marks	4 Marks

Solution Architecture

Solution architecture is a complex process - with many sub-processes - that bridges the gap between business problems and technology solutions. Its goals are to:

• Find the best tech solution to solve existing business problems.

• Describe the structure, characteristics, behavior, and other aspects of the software to project stakeholders.

• Define features, development phases, and solution requirements.

• Provide specifications according to which the solution is defined, managed, and delivered.

The ShopEZ Stock Trading Platform implements a 3-tier MERN stack architecture to separate user interfaces, business logic, and cloud database persistence:

• **Client Tier**: Single Page Application (SPA) built using React, Vite, Chart.js, and standard CSS pastel variables, communicating via Axios.

• **Application Tier**: Express.js REST API server running on Node.js. Handles routes, controllers, JWT checks, validations, and the simulation ticks engine.

• **Data Tier**: MongoDB and MongoDB Atlas database storing transaction logs, listings, portfolios, and user credentials.

Components & Database Schema

User Model

Stores core trader/admin authentication, credentials, and virtual cash balances:

```javascript
```

name: type: String, required: true ,

email: type: String, required: true, unique: true ,

password: type: String, required: true, select: false ,

role: type: String, enum: ['Trader', 'Admin'], default: 'Trader' ,

profilePic: type: String, default: '' ,

balance: type: Number, default: 100000 , // Virtual starting cash ($100k)

timestamps: true

```

### Stock Model

Stores listings, names, current prices, starting values, and price history:

```javascript

ticker: type: String, required: true, unique: true ,

name: type: String, required: true ,

currentPrice: type: Number, required: true ,

startingPrice: type: Number, required: true ,

history: [ type: Number ], // Historical ticks for line chart

timestamps: true

```

Transaction Model

Tracks order executions (BUY/SELL) logged for auditing:

```javascript

user: type: Schema.Types.ObjectId, ref: 'User', required: true ,

ticker: type: String, required: true ,

type: type: String, enum: ['BUY', 'SELL'], required: true ,

shares: type: Number, required: true ,

price: type: Number, required: true ,

timestamp: type: Date, default: Date.now

```

Portfolio Model

Tracks current asset holdings for each trader:

```javascript

user: type: Schema.Types.ObjectId, ref: 'User', required: true ,

ticker: type: String, required: true ,

sharesOwned: type: Number, default: 0 ,

averageCost: type: Number, default: 0

```

Security Architecture

1. **Password Hashing**: Done pre-save inside the User schema using the `bcryptjs` hashing library.

2. **Access Control (Backend)**: Custom middleware handlers secure requests:

- `protect`: Extracts the Bearer token from request authorization headers, validates it against the platform's `JWT_SECRET`, and attaches the authenticated User model to `req.user`.
- `authorize(...roles)`: Verifies if the request initiator's role matches permitted roles (e.g., restricting stock insertions to `Admin`).

3. **Route Guards (Frontend)**: Custom React `ProtectedRoute` wrappers that query the `AuthContext` to redirect unauthenticated requests back to `/login`.
```